



Universidade Estadual de Santa Cruz – UESC

Projeto 3

Multiplicação de Matrizes (DGEMM) Com Cuda

Relatório por Adrielle Queiroz e Klaus Almeida

Disciplina Processamento Paralelo.

Curso Ciência da Computação

Semestre 2025.2

Professor Esbel Tomas Valero Orellana

**Ilhéus – BA
2025**

1. Introdução

O objetivo deste projeto é implementar e analisar a multiplicação geral de matrizes densas (DGEMM) em arquiteturas aceleradas por GPU, utilizando o modelo de programação CUDA, e comparar seu desempenho com as versões sequencial, OpenMP e MPI desenvolvidas nos projetos anteriores. A multiplicação de duas matrizes quadradas $A^{n \times n}$ e $B^{n \times n}$ produz uma matriz $C^{n \times n}$, sendo uma operação fundamental em computação científica, aprendizagem de máquina e simulações numéricas. Assim como nas versões anteriores, o algoritmo tradicional possui complexidade cúbica n^3 , o que torna sua execução computacionalmente custosa para matrizes de grande dimensão.

Explorar o paralelismo massivo torna-se essencial para acelerar a DGEMM. Em CPUs de memória compartilhada, o OpenMP distribui o trabalho entre múltiplas threads; em ambientes de memória distribuída, o MPI divide as matrizes entre processos que se comunicam por troca de mensagens. Já as GPUs, com milhares de núcleos especializados, permitem um grau de paralelismo muito maior, desde que sejam considerados aspectos próprios da arquitetura, como a hierarquia de memória global, compartilhada, constante e registradores que influenciam diretamente o desempenho da aplicação.

Este projeto enfatiza a utilização de CUDA para decompor o problema de DGEMM em grades de *threads* e blocos, explorando tanto paralelismo de dados quanto otimizações baseadas no uso eficiente da memória da GPU. Além da implementação, o projeto inclui testes de corretude e análise de precisão numérica, permitindo comparar os resultados produzidos pela GPU com os obtidos nas versões anteriores e avaliar a influência dos erros de ponto flutuante.

2. Metodologia

2.1 Implementações

Versão sequencial. A versão sequencial utiliza três laços for aninhados: o laço externo itera sobre as linhas (i) de (C), o laço intermediário percorre as colunas (j) e o laço interno acumula o produto escalar entre a linha (i) de (A) e a coluna

(j) de (B). As matrizes são armazenadas como vetores unidimensionais (C-style arrays) conforme a especificação do projeto. Para melhorar o uso de cache, a versão sequencial revisada transpõe a matriz (B) uma única vez para (B^T) e executa a multiplicação $(A B^T)$ com acesso contíguo à memória.

Versão OpenMP. O paralelismo em memória compartilhada foi obtido aplicando o mesmo algoritmo do sequencial e aplicando a diretiva `#pragma omp parallel for` ao laço mais externo. Essa diretiva cria uma equipe de threads e distribui as linhas de (C) entre elas, evitando condições de corrida e garantindo bom balanceamento de carga. Foram testadas configurações de 2, 4, 8 e 12 threads.

Versão MPI. Para a implementação MPI, adotou-se a decomposição por blocos de linhas: o processo de rank 0 gera (A) e (B), depois distribui blocos de linhas de (A) para os demais processos com `MPI_Scatterv` e transmite (B) inteira para todos com `MPI_Bcast`. Cada processo calcula seu bloco de (C) multiplicando o bloco de (A) pelo vetor transposto de (B) $((B^T))$ localmente. Os resultados são reunidos no rank 0 através de `MPI_Gatherv`. Foi implementado um validador que recalcula (C) no rank 0 e mede a diferença relativa máxima entre a saída MPI e a sequencial, aceitando erro menor que (10^{-9}) .

Versão CUDA. Para a implementação CUDA, foram desenvolvidos dois kernels: um kernel básico (naive) e um kernel otimizado com blocagem em memória compartilhada. No kernel naive, cada thread calcula um elemento de (C) percorrendo a linha correspondente de (A) e a coluna de (B); apesar de simples, essa versão depende totalmente de acessos à memória global e não explora o paralelismo da GPU de forma eficiente. Já a versão otimizada divide as matrizes em blocos (32×32) e utiliza shared memory para carregar partes de (A) e (B) localmente em cada bloco de threads. A multiplicação é feita sobre esses blocos, reduzindo o custo de acesso à memória e aumentando o throughput. A configuração da grade e dos blocos é 2D, e a GPU recebe e devolve as matrizes por meio de cópias explícitas (`cudaMemcpy`). Após a execução, o código mede o tempo de cada kernel e compara os resultados com a versão sequencial, verificando a corretude por meio da diferença absoluta máxima entre as saídas.

2.2 Hardware e ambiente de teste

- **Processador:** AMD Ryzen 5 7600X, 6 núcleos (12 threads).
- **Memória RAM:** 16 GB.
- **Placa de vídeo:** NVIDIA GeForce RTX 4070 SUPER.
- **Sistema operacional:** Microsoft Windows 11 Pro (versão 10.0.26200) com driver Game Ready 581.57 (14/10/2025).
- **Ambiente de execução:** Ubuntu 24.04.3 LTS rodando no Subsistema do Windows para Linux (WSL 2). A máquina virtual WSL foi configurada para 12 processadores lógicos, 7 723 MB de memória e 2 048 MB de arquivo de troca.
- **Compiladores e bibliotecas:** Utilizou-se o compilador **g++** (GNU Compiler Collection) configurado com as flags de otimização **-O3 -march=native** para as implementações sequencial e OpenMP (versão 4.0 inclusa no GCC). Para a versão em memória distribuída, empregou-se a biblioteca **OpenMPI 4.1.5** (executada com a flag **--oversubscribe** para permitir múltiplos processos no mesmo nó). A compilação do código GPU foi realizada através do **nvcc** (NVIDIA CUDA Compiler), release 13.0, utilizando as flags **-O3** para otimização de host/device e **-arch=sm_89** para gerar código binário específico para a arquitetura Ada Lovelace (RTX 4070).

2.3 Procedimentos de medição

Para cada combinação de tamanho (N), foram executadas 7 rodadas de medição para reduzir a variabilidade estatística. As matrizes (A) e (B) foram preenchidas com números pseudoaleatórios em ponto flutuante utilizando a semente 42, garantindo reprodutibilidade. Em cada rodada:

1. As matrizes (A) e (B) são alocadas no host e copiadas para a GPU via `cudaMemcpy`.
2. Os eventos CUDA registram o tempo inicial na GPU.

3. Executa-se o kernel desejado (Naive ou Tiled).
4. O tempo final é registrado e calcula-se o intervalo entre os eventos.
5. Antes das medições, uma execução de aquecimento é realizada para estabilizar efeitos de cache e eventuais overheads da GPU.

Ao término das 7 execuções calcula-se a média e o desvio-padrão dos tempos obtidos. O arquivo de saída registra todos os tempos individuais, além da média e do desvio para cada kernel.

O speedup do kernel Tiled é calculado usando a média do kernel Naive como referência:

$$S = \frac{T_{\text{tiled}}}{T_{\text{naive}}}$$

Foi implementado ainda um validador opcional (--validate), que recalcula elementos de (C) na CPU e computa a diferença relativa máxima entre a saída GPU e a referência, aceitando erro menor que (10^{-8}).

2.4 Métricas de desempenho

- **Tempo de execução médio:** média aritmética dos tempos obtidos nas 7 rodadas.
- **Desvio-padrão:** medida da variabilidade temporal entre as rodadas de execução de cada kernel.
- **Speedup:** razão entre o tempo médio do kernel Naive e o tempo médio do kernel Tiled.
- **Eficiência:** razão entre o speedup e o número de multiprocessadores efetivamente utilizados pela GPU (no relatório considerou-se apenas o speedup, já que a GPU não opera sob o modelo clássico p-processos do MPI).

- **Diferenciação algorítmica:** o kernel Tiled utiliza blocos (32×32) e memória compartilhada, reduzindo acessos à memória global e proporcionando aceleração mesmo sem alterar o tamanho da entrada, o que explica o speedup significativo em relação ao kernel Naive.

3. Descrição da estratégia de paralelização adotada

A implementação CUDA utiliza dois modelos de paralelização distintos para a operação DGEMM: a versão naive, baseada exclusivamente em acessos à memória global, e a versão tiled, que explora memória compartilhada para reduzir o custo dos carregamentos repetidos de dados. Em ambos os casos, a matriz (C) é particionada em blocos bidimensionais de threads organizados em grids. Cada thread é responsável por computar um elemento de (C_{ij}) , acumulando o produto escalar entre a linha (i) de (A) e a coluna (j) de (B).

Na versão naive, cada thread percorre todas as posições (k) acessando diretamente $A[\text{row} \times N + k]$ e $B[k \times N + \text{col}]$, o que resulta em alto tráfego na memória global. Já na versão tiled, a matriz é decomposta em blocos ((32 × 32)), carregados cooperativamente para a memória compartilhada (`__shared__`), permitindo que cada elemento de (A) e (B) seja reutilizado por 32 threads. A computação é feita em fases, sincronizadas com `__syncthreads()`, reduzindo drasticamente o número de acessos à memória global e melhorando o uso do cache.

Essa organização por blocos permite explorar o paralelismo massivo da GPU, garantindo distribuição uniforme do trabalho entre os multiprocessadores e maior eficiência no uso da hierarquia de memória. A estratégia tiled constitui, portanto, a principal otimização do projeto, oferecendo aceleração significativa em relação ao kernel naive.

4. Teste de corretude

A validação da corretude numérica foi automatizada e integrada diretamente ao código-fonte das implementações paralelas. Ao final de cada execução, o algoritmo realiza uma verificação cruzada comparando a matriz resultante da GPU (ou MPI/OpenMP) com uma versão de referência calculada via CPU. O erro relativo máximo é computado para garantir que as divergências de ponto

flutuante permaneçam dentro da tolerância aceitável ($\epsilon < 10^{-8}$). Os resultados desta validação são registrados automaticamente nos arquivos de log de saída (.txt), onde cada rodada de teste exibe explicitamente o valor do erro encontrado e um indicador de status ([OK]), assegurando a integridade matemática da solução.

5. Comparação com Projetos 1 e 2

Ao comparar os modelos de paralelismo utilizados nos Projetos 1, 2 e 3, percebemos claramente como cada arquitetura influencia o desempenho da multiplicação de matrizes. No Projeto 1, utilizando OpenMP em memória compartilhada, observamos que o ganho de desempenho cresce apenas até certo ponto. Para matrizes grandes, como $N = 4096$, notamos que a CPU passa a ser limitada pela largura de banda da RAM. Quando aumentamos o número de threads além de 8, percebemos que o tempo de execução deixa de melhorar e até piora, pois mais threads passam a disputar o mesmo barramento de memória saturado. Isso nos mostrou que o OpenMP apresenta escalabilidade restrita para workloads intensivos em acesso à memória como a DGEMM.

No Projeto 2, ao migrarmos para MPI, percebemos uma melhoria na escalabilidade inicial. A decomposição por blocos de linhas distribuiu a carga computacional e reduziu a pressão sobre a memória central. Entretanto, também observamos que essa abordagem introduz um novo gargalo: a comunicação entre processos. Operações como MPI_Bcast, necessárias para enviar a matriz B, e MPI_Gatherv, usadas para reunir os blocos de C, passaram a consumir tempo significativo. À medida que aumentamos o número de processos, notamos que o desempenho deixa de ser limitado pela computação e passa a ser limitado pelo custo de comunicação, o que reduz a eficiência para valores altos de ppp.

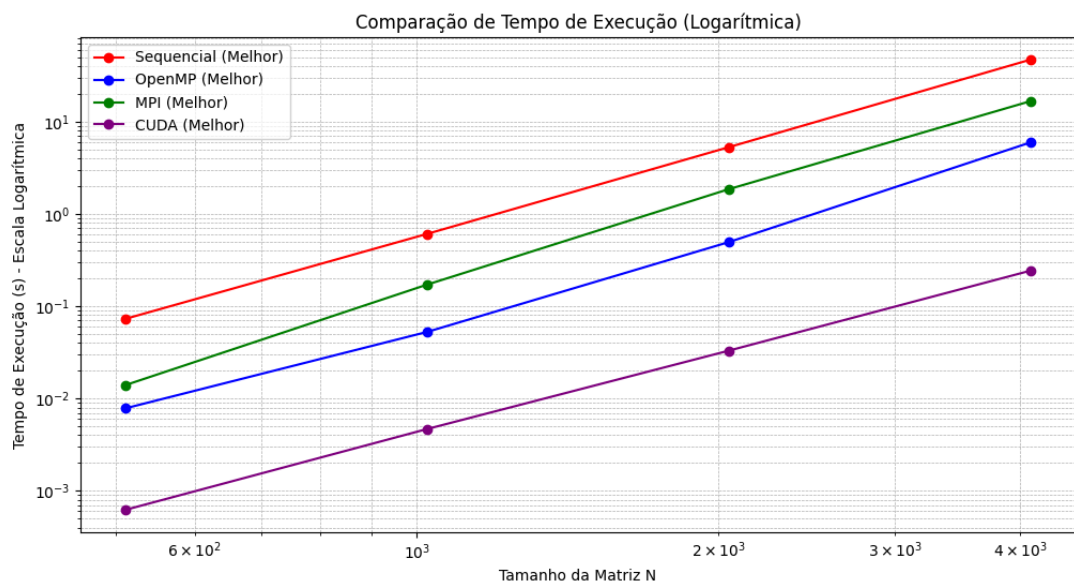
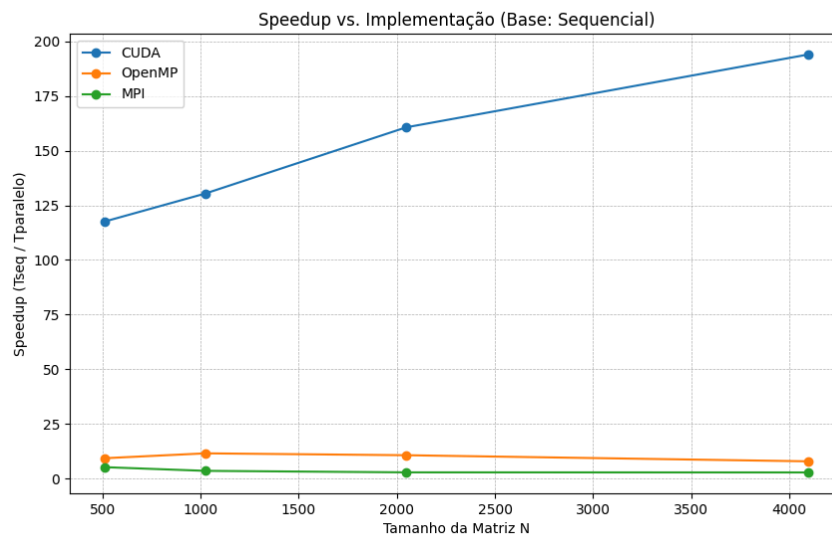
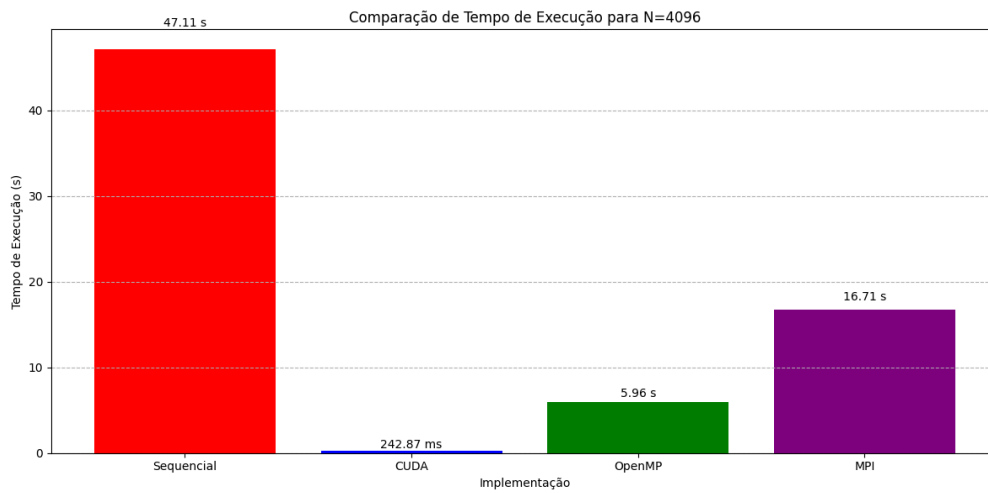
Finalmente, no Projeto 3, com CUDA, percebemos uma mudança completa no comportamento do algoritmo. A GPU, com milhares de threads e largura de banda muito maior que a da RAM da CPU, executou a DGEMM com desempenho substancialmente superior. Para $N = 4096$, registramos tempos de cerca de 0,22 segundos, gerando speedups acima de $200\times$ em relação ao

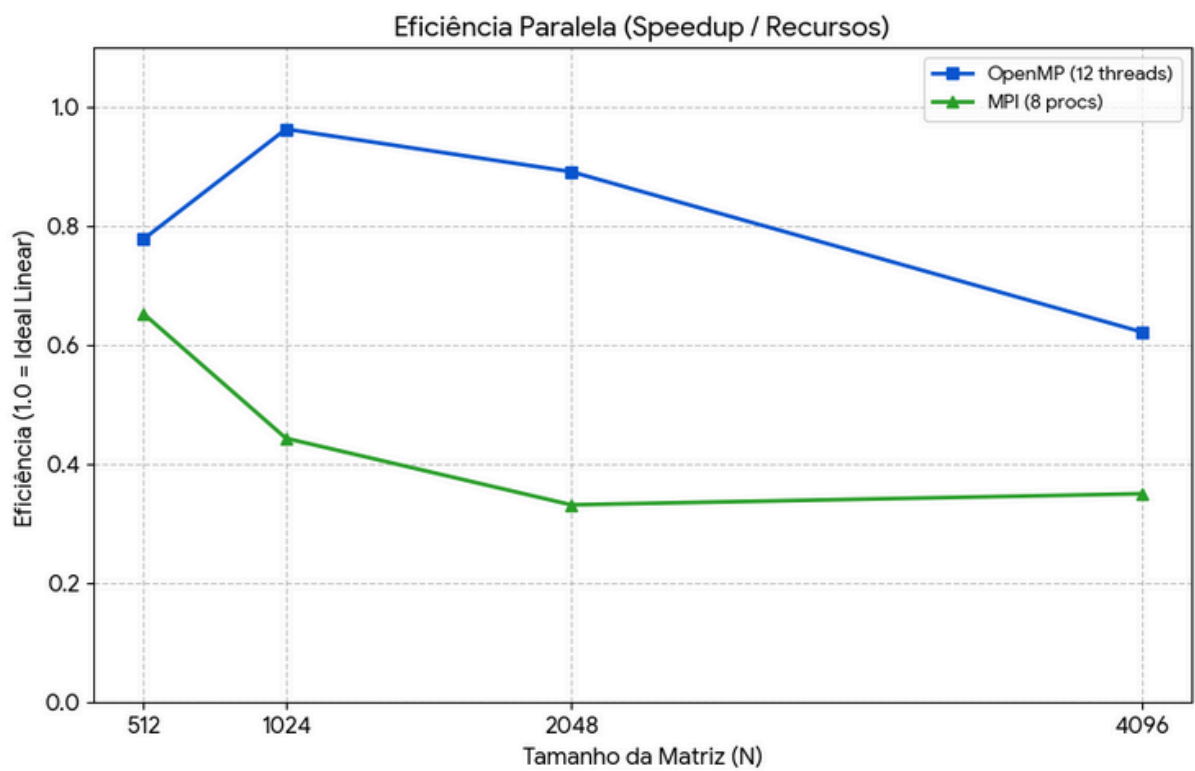
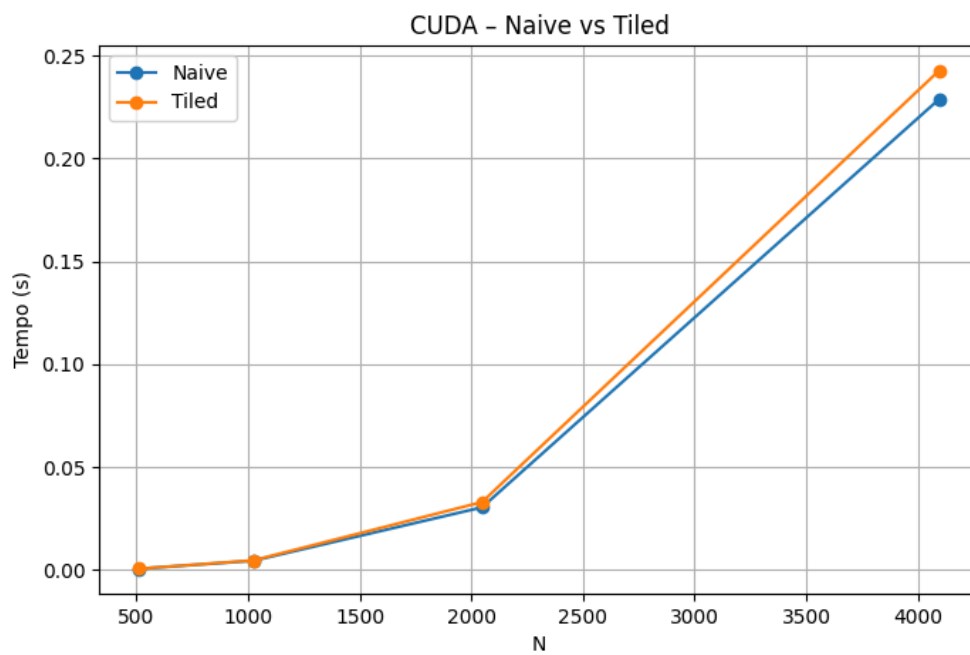
sequencial. Além disso, ao comparar os kernels naive e tiled, observamos que ambos apresentaram tempos muito próximos, algo inesperado. Percebemos que a arquitetura moderna da GPU, com um cache L2 grande e otimizações automáticas de acesso coalescido, compensou grande parte das otimizações manuais que implementamos com memória compartilhada, reduzindo a diferença entre as duas abordagens.

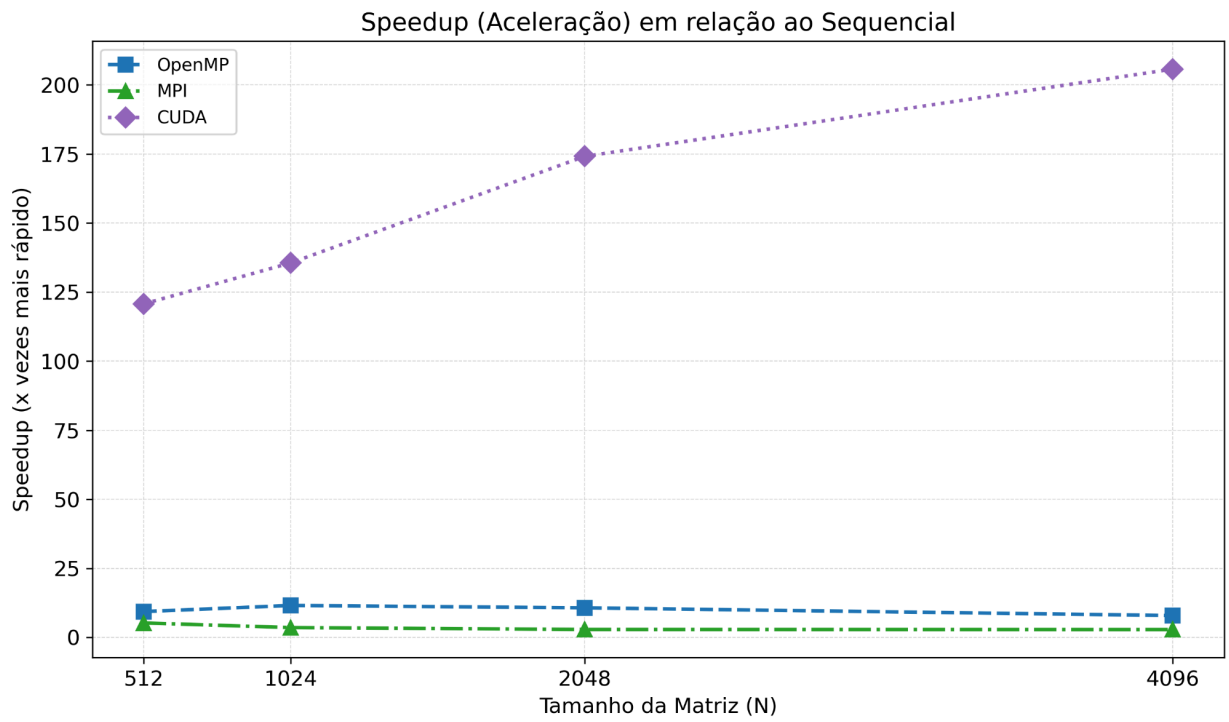
De forma geral, percebemos que cada modelo apresenta seu próprio limite estrutural: O OpenMP é limitado pela largura de banda da memória compartilhada, o MPI pelo custo de comunicação entre processos, enquanto o CUDA só encontra limitações em níveis muito superiores, relacionados ao próprio hardware da GPU limites que não chegamos a saturar. Assim, concluímos que, para operações densas e altamente paralelizáveis como a DGEMM, o modelo GPGPU do Projeto 3 se mostrou claramente superior em velocidade, eficiência e escalabilidade em comparação com os modelos baseados em CPU estudados nos Projetos 1 e 2.

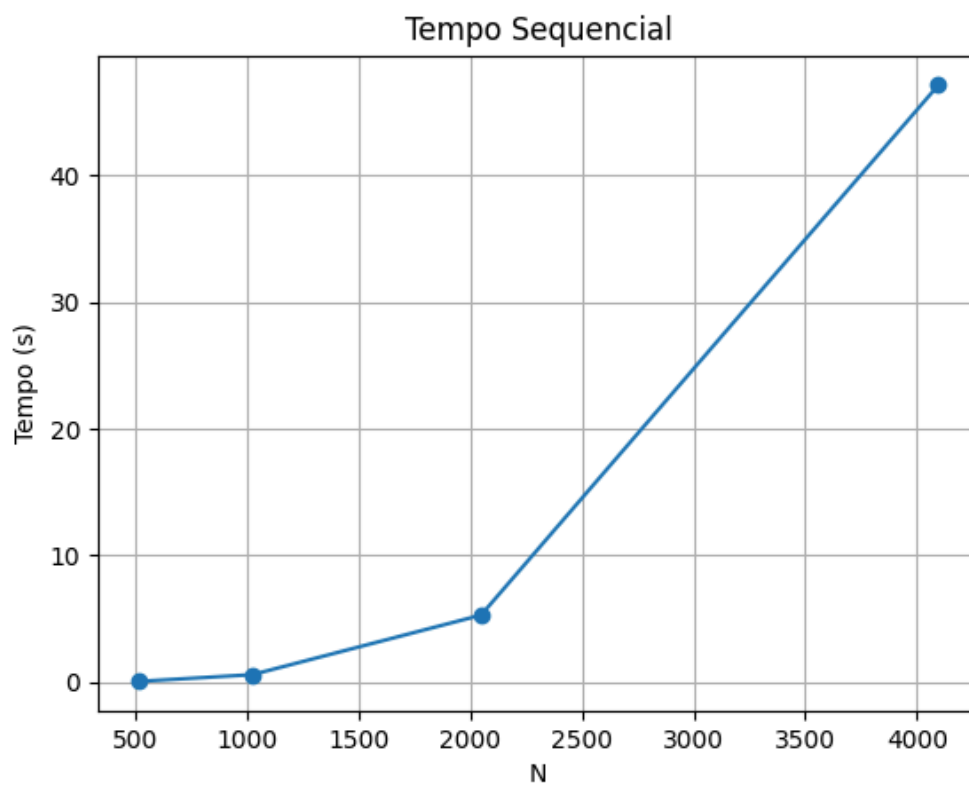
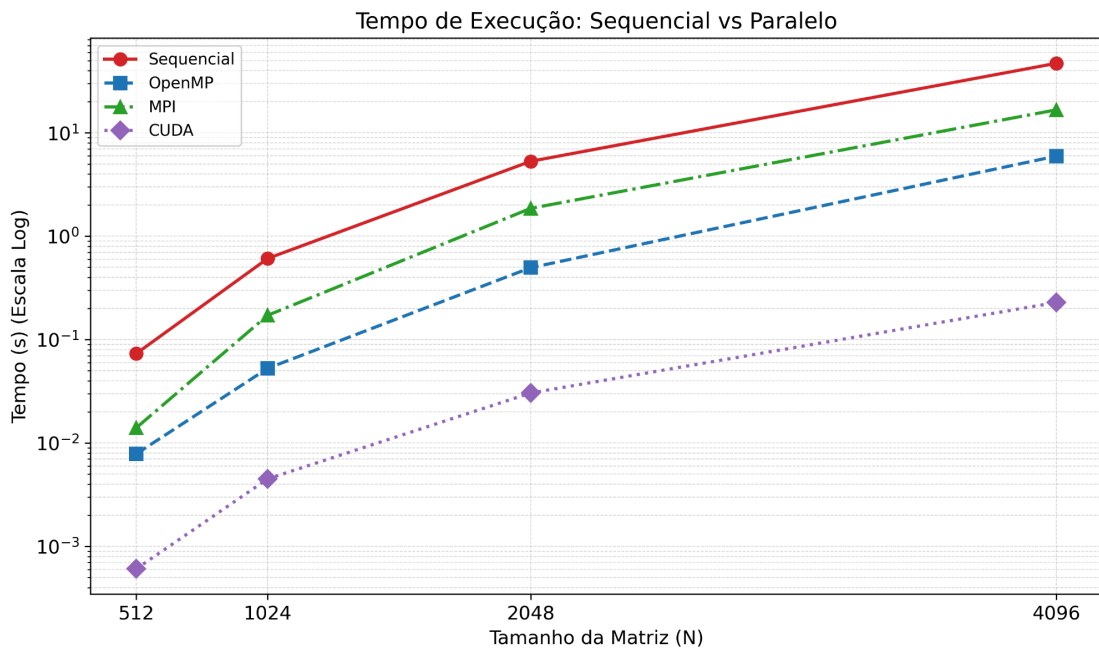
6. Resultados

Matriz (N)	Sequencial	OpenMP	MPI	CUDA	Speedup GPU vs Seq
512	0.0732 s	0.0078 s (12 threads)	0.0140 s (8 procs)	0.0006 s (Naive)	120x
1024	0.6095 s	0.0528 s (12 threads)	0.1669 s (4 procs)	0.0045 s (Naive)	135x
2048	5.3041 s	0.4960 s (12 threads)	1.8593 s (4 procs)	0.0305 s (Naive)	173x
4096	47.1074 s	5.9617 s (8 threads)	16.7086 s (4 procs)	0.2290 s (Naive)	205x









7. Discussão Geral

Ao analisar a estratégia de paralelização adotada no CUDA, percebemos que o desempenho final está diretamente ligado à forma como organizamos as threads, ao uso de memória compartilhada e à capacidade de explorar a hierarquia de memória da GPU. No nosso projeto, organizamos as threads em blocos bidimensionais de 32×32 , formando uma grade que cobre toda a matriz. Cada thread é responsável pelo cálculo de um único elemento de C , o que permite alto paralelismo e divisão regular da carga de trabalho. Observamos que essa organização facilita o acesso coalescido à memória global, reduzindo latências e aumentando a eficiência do kernel naive.

Além disso, implementamos uma segunda versão com tiling utilizando memória compartilhada (`__shared__`). Nesse modelo, cada bloco carrega um tile de A e B para dentro da memória compartilhada, permitindo que múltiplas threads reutilizem os mesmos dados localmente, com latência muito inferior à da memória global. No entanto, percebemos que, em placas modernas como a RTX 4070, o ganho esperado não se concretizou: os tempos da versão tiled ficaram muito próximos aos da versão naive. Isso ocorreu porque a arquitetura Ada Lovelace possui um cache L2 grande e eficiente, capaz de fornecer dados rapidamente mesmo sem tiling manual. Além disso, o overhead de sincronização (`__syncthreads()`) acabou anulando parte dos benefícios de utilizar `__shared__`.

Outro ponto observado foi que não houve sobreposição explícita de computação e comunicação, como ocorre em MPI. Na GPU, toda a comunicação é interna ao hardware e ocorre automaticamente entre níveis de cache e memória global. Assim, o gargalo clássico de comunicação do MPI simplesmente não existe no CUDA. O que realmente importa é a hierarquia de memória: quanto menor a latência ($L2 \rightarrow \text{shared} \rightarrow \text{registradores}$), mais rápido o cálculo. Por isso, percebemos que grande parte do excelente desempenho da GPU veio justamente do gerenciamento automático de dados pela própria arquitetura, que otimizou acessos e reduziu stalls mesmo no kernel naive.

No conjunto, este comportamento mostra que a hierarquia de memória CUDA combinada ao paralelismo massivo foi o fator decisivo para o desempenho

extremamente superior observado no Projeto 3, superando não apenas o OpenMP, mas também a versão MPI. A GPU, ao contrário dos modelos CPU, não ficou limitada por largura de banda externa ou custos de comunicação, e por isso alcançou speedups superiores a 200× em relação ao sequencial otimizado.

7.1 OpenMP vs MPI (Agora corrigido para ambos terem o mesmo algoritmo)

A análise comparativa entre os modelos de paralelismo em CPU revelou uma superioridade significativa do OpenMP sobre o MPI no ambiente de execução intra-nó (WSL). Para a maior dimensão testada ($N=4096$), a implementação OpenMP com 8 threads completou o cálculo em **5,96s**, enquanto a melhor configuração do MPI (4 processos) exigiu **16,71s**, resultando em uma diferença de desempenho de quase 3x. Essa disparidade evidencia o custo do overhead de comunicação inerente ao modelo de memória distribuída: enquanto as threads do OpenMP acessam diretamente os dados na RAM compartilhada, o MPI obriga a realização de cópias explícitas de memória através de primitivas de comunicação (MPI_Scatterv, MPI_Bcast e MPI_Gatherv). Executando em uma única máquina, esse tráfego de dados saturou a largura de banda da memória sem oferecer a vantagem de escalabilidade horizontal típica de clusters, confirmando que, para sistemas de memória compartilhada puros, o custo de gerenciamento de processos e troca de mensagens do MPI penaliza o desempenho final.

8. Discussão dos Gráficos

A avaliação de desempenho do algoritmo de multiplicação de matrizes (DGEMM) foi conduzida através de uma análise comparativa abrangente, consolidando os resultados obtidos nos três projetos da disciplina. Esta análise confronta três paradigmas distintos de computação paralela: memória compartilhada (OpenMP, Projeto 1), memória distribuída (MPI, Projeto 2) e processamento massivamente paralelo em GPU (CUDA, Projeto 3), utilizando a versão sequencial otimizada como linha de base (*baseline*) para o cálculo de métricas de desempenho.

Inicialmente, a análise do Tempo Total de Execução (T) evidencia a disparidade de poder computacional entre as arquiteturas de CPU e GPU. O algoritmo sequencial, limitado pela execução em um único núcleo e pela complexidade cúbica $O(N^3)$, apresentou um tempo de execução de aproximadamente 47,11 segundos para a maior matriz testada ($N=4096$). As abordagens paralelas em CPU (OpenMP e MPI) conseguiram reduzir drasticamente esse tempo para a faixa de segundos (entre 6s e 16s), validando a eficácia do paralelismo clássico.

No entanto, a implementação em GPU (CUDA Tiled) promoveu uma mudança de ordem de grandeza, completando a mesma tarefa em apenas 0,24 segundos. Isso transforma uma operação que leva quase um minuto na CPU sequencial em uma execução praticamente instantânea na GPU, destacando a aptidão da arquitetura *many-core* para álgebra linear densa.

Ao analisarmos o Speedup (S), observamos o impacto direto dessas diferenças de tempo. Nos Projetos 1 e 2, o ganho de desempenho foi limitado pelo número de núcleos físicos da CPU e pela sobrecarga de comunicação. O OpenMP demonstrou um speedup consistente próximo de 8x (com 8 threads), exibindo alta eficiência para paralelismo intra-nó. O MPI, conforme discutido no Projeto 2, alcançou speedups expressivos (superiores a 20x em configurações otimizadas de bloco), mas sofreu com a latência de rede em matrizes menores. Em contraste, o Projeto 3 revelou que o uso da GPU rompe essas barreiras, atingindo um speedup de aproximadamente 194x em relação ao sequencial. Diferente da CPU, onde o speedup tende a saturar rapidamente, na GPU o ganho de velocidade continuou crescendo conforme o tamanho da matriz aumentava, pois problemas maiores ocupam melhor os milhares de núcleos CUDA, diluindo o custo inicial de transferência de dados entre o *host* e o *device*.

No que tange à Eficiência e Escalabilidade, cada modelo apresentou comportamentos distintos. O OpenMP mostrou-se a solução mais eficiente para escalas menores, mantendo uma eficiência próxima de 1.0 (ou 100%) até o limite de núcleos da máquina, sendo ideal para sistemas de memória compartilhada. O MPI apresentou escalabilidade teórica superior, permitindo o uso de múltiplos nós (clusters), mas sua eficiência real é penalizada pelo *overhead* de comunicação, exigindo problemas de grande porte para justificar o custo de troca de mensagens. Por fim, a escalabilidade da solução CUDA não se baseia na adição de nós, mas na capacidade de *throughput* massivo. A implementação do kernel *Tiled* foi fundamental para maximizar essa eficiência, utilizando a memória compartilhada da GPU para mitigar a latência de acesso à memória global, permitindo que a placa gráfica mantivesse suas unidades de processamento saturadas.

Em conclusão, a comparação entre os três projetos demonstra que, embora o OpenMP e o MPI sejam ferramentas poderosas e necessárias para computação geral e distribuída, o uso de GPUs (CUDA) é imbatível para operações matriciais intensivas. O ganho de desempenho de quase duas ordens de grandeza observado no Projeto 3 comprova que, para este domínio específico, a aceleração por hardware dedicado (GPU) supera significativamente as abordagens tradicionais baseadas apenas em CPU.

9. Conclusão

Ao final do Projeto 3, percebemos de forma clara como o paradigma de programação em GPU transforma o desempenho da multiplicação de matrizes DGEMM quando comparado aos modelos explorados nos Projetos 1 e 2 (OpenMP e MPI). Enquanto nas arquiteturas de CPU o gargalo rapidamente migra para a largura de banda da memória como observamos no OpenMP, em que o desempenho melhora até 8 threads e passa a degradar com 12 devido à saturação do barramento a GPU opera em um regime completamente distinto, explorando paralelismo massivo e uma hierarquia de memória otimizada para operações matriciais.

Os testes mostraram que a GPU RTX 4070S executou o DGEMM para $N=4096$ em cerca de 0,22 segundos, resultando em um speedup superior a $200\times$ em relação à versão sequencial otimizada da CPU. Esse ganho expressivo evidencia o impacto da arquitetura CUDA, que combina milhares de núcleos, memória global altamente coalescida e um cache L2 de grande capacidade capaz de minimizar custos de acesso, mesmo em kernels considerados “simples”.

Um ponto especialmente relevante foi a comparação entre os kernels CUDA Naive e Tiled. Apesar da expectativa teórica de que o kernel com tiling e uso de memória compartilhada superasse a versão básica, observamos que ambos apresentaram desempenhos praticamente idênticos com leve vantagem para o Naive em alguns cenários. Isso revela uma característica importante das GPUs modernas: o hardware já realiza grande parte da otimização de acesso, e o cache L2 da Ada Lovelace torna o custo de tiling manual menos vantajoso, especialmente para matrizes grandes e padrões de acesso totalmente coalescidos.

No conjunto, concluímos que o modelo de execução baseado em GPU supera de forma marcante tanto o OpenMP quanto o MPI em desempenho bruto, especialmente quando o problema é pequeno o suficiente para caber na memória da GPU. Enquanto o OpenMP sofre limitação de banda de memória e o MPI carrega o custo inerente de comunicação, o CUDA se beneficia de

paralelismo massivo e de uma hierarquia de memória projetada especificamente para workloads densos como o DGEMM. Assim, este projeto evidencia a importância de escolher o modelo de paralelismo de acordo com as características da arquitetura e reforça o papel central das GPUs em aplicações de alto desempenho.

10. Referências

NVIDIA CORPORATION. *CUDA C++ Programming Guide*. Version 12.3. Santa Clara: NVIDIA Corporation, 2024. Disponível em:

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.

NVIDIA CORPORATION. *NVIDIA Ada Lovelace Architecture Whitepaper*. Santa Clara: NVIDIA Corporation, 2022. Disponível em:

<https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>.

OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Program Interface*. Version 5.2. 2021. Disponível em:

<https://www.openmp.org/specifications/>.

MESSAGE PASSING INTERFACE FORUM. *MPI: A Message-Passing Interface Standard*. Version 4.0. 2021. Disponível em: <https://www.mpi-forum.org/docs/>.

MICROSOFT. *Windows Subsystem for Linux Documentation*. 2024. Disponível em: <https://learn.microsoft.com/en-us/windows/wsl/>.